Apps Script & AppSheet Work-Transformation Governance

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

The opportunity and the risk . . . 4

What this solution establishes . . . 4

How it's delivered . . . 4

The bottom line . . . 4

Solution Overview 5

Apps Script & AppSheet Work-Transformation Governance . . . 5

Requirements 6

1. Super-admin (or delegated) access to the Google Admin console . . . 7

2. AppSheet administration access and edition awareness . . . 7

3. A data-governance baseline (data classification + connected-source inventory) . . . 7

4. High-risk OAuth-scope restriction capability (edition-gated) . . . 8

5. A named review model (roles + checklist) . . . 8

6. A pilot OU/group and change control . . . 8

Global Requirements — Google Workspace & Gemini Solutions 9

1. Google Workspace / Cloud Identity tenant . . . 9

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 9

3. DNS control for the routed domains . . . 9

4. Identity & provisioning source of truth . . . 10

5. A pilot scope (OU / group) and change control . . . 10

6. Security, compliance & governance stakeholders . . . 10

Implementation 11

Phase 0 — Assess (inventory the estate) . . . 11

Phase 1 — Configure the admin guardrails (platform floor) . . . 11

Phase 2 — Define the approval/review pattern and environments . . . 13

Phase 3 — Pilot . . . 14

Phase 4 — Validate and Expand . . . 14

Testing 14

Gate 1 — App access control posture (Apps Script data access) . . . 14

Gate 2 — High-risk OAuth scope restriction (edition-gated) . . . 15

Gate 3 — AppSheet external sharing blocked . . . 15

Gate 4 — AppSheet external data sources / webhooks / external API restricted . . . 15

Gate 5 — Require sign-in . . . 15

Gate 6 — Deploy control (review model is enforceable) . . . 16

Gate 7 — Owner of record and environment discipline . . . 16

Gate 8 — Builders can still ship (guardrails aren't a lockdown) . . . 16
Conformance summary . . . 16
Troubleshooting 16
Settings changed but nothing seems different . . . 17
Builders suddenly can't run apps they relied on . . . 17
Apps Script authorization / scope errors after restriction . . . 17
AppSheet policy isn't enforcing . . . 17
Core security setting is too blunt . . . 18
"Anyone can mark apps deployed" — review model not enforced . . . 18
Discovery/Validation returns nothing or errors . . . 18
Data-residency requirement not met . . . 18
Remediation would change more than expected . . . 18
Best Practices — Safe Citizen Development 19
Least scope, always . . . 19
Internal-only by default; external sharing is a decision . . . 19
Require sign-in and know the user . . . 19
Owner of record on everything . . . 19
Build → Pilot → Production, never straight to prod . . . 20
Reuse the governed baseline . . . 20
Tier the review, don't gate everything . . . 20
A one-page review checklist . . . 20
Keep the inventory alive . . . 20
Change under control; propagation takes time . . . 20
Deliverables Summary 21
Documentation . . . 21
Scripts (5) . . . 21
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary

The opportunity and the risk

Google Workspace gives every team two ways to redesign work at speed: **Apps Script** (automation and add-ons across Gmail, Drive, Sheets, and the Workspace APIs) and **AppSheet** (no-code apps and workflow automation on top of your data). They let the business rebuild a workflow in hours instead of quarters. Left ungoverned, the same speed produces **shadow IT** — scripts and apps that read and move corporate data on broad OAuth scopes, share outside the organization, and have no owner, no review, and no rollback.

What this solution establishes

A **work-transformation governance model** that keeps the speed and removes the exposure. It targets the three places the risk actually concentrates — **data access** (OAuth scopes and connected sources), **sharing** (who can run an automation and see its data), and **app access itself** (can users build/run these at all) — and channels each with a layered control:

- **Admin guardrails (the floor).** In the Google Admin console: Apps Script **app access control**

(trusted/limited/blocked and high-risk OAuth-scope restriction) and the **AppSheet** security setting or **Enterprise Plus governance policies** (require sign-in, restrict external sources, block external sharing, control who can deploy, and data region).

- **Approval & review pattern (the process).** A tiered review — low-risk automations self-serve;

shared, broad-scope, or regulated builds go through a named reviewer against a one-page checklist before they can be marked deployed.

- **Environment & sharing standards (the structure).** Named **Build** → **Pilot** → **Production**

environments, internal-only sharing defaults, an owner of record on every artifact, a rollback path, and a reuse baseline of blessed templates and shared libraries.

How it's delivered

Self-guided and pilot-first, following **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with explicit pass/fail gates and documented rollback. Seven guides plus five PowerShell scripts — read-only **Discovery**, **Validation**, and **Report** for inventory, conformance, and risk; a default-preview (`-WhatIf`) **Remediation** helper; and a **Set-ScopeTargeting** scope helper — turn "we have Apps Script and AppSheet out there somewhere" into a governed, inventoried, reviewed capability.

The bottom line

Business-led workflow automation that is fast **and** safe — Apps Script and AppSheet governed with guardrails, approvals, and standards — deployed to a pilot first and rollback-ready. Citizen and pro developers redesign workflows to enterprise standard, at a fraction of the cost of bespoke consulting, without turning speed into shadow IT.

Solution Overview

Apps Script & AppSheet Work-Transformation Governance

Google Workspace ships two of the most powerful low-friction automation platforms in the enterprise: **Apps Script** (script-based automation and add-ons across Gmail, Drive, Sheets, Docs, Calendar, and the Workspace APIs) and **AppSheet** (no-code apps and workflow automation built directly on Sheets, databases, and connected data sources). Both let a business team redesign a workflow in hours instead of quarters — and both, ungoverned, become **shadow IT**: scripts and apps that read and move corporate data, run on broad OAuth scopes, and share outside the organization with no review, no owner of record, and no rollback plan.

This solution establishes a **work-transformation governance model** for Apps Script and AppSheet so that citizen developers (business users) and pro developers (IT/engineering) can redesign workflows **safely and to a repeatable enterprise standard**. It is not a lockdown — it is a set of guardrails, an approval and review pattern, and environment/sharing standards that channel the speed of these platforms without exposing data.

Where the risk actually is

Governance is only useful if it targets the real exposure. For Apps Script and AppSheet, the risk concentrates in three places:

1. **Data access (OAuth scopes and connected sources).** An Apps Script project runs with the

authorizing user's OAuth grants; a broad scope (for example, full Drive or full Gmail) lets the script read or move anything that user can. AppSheet apps connect to Sheets, Cloud SQL, and third-party sources and inherit that data. The guardrail question is: *what data can this automation touch, and did anyone approve that reach?*

1. **Sharing (who can run it, who can see the data).** Apps Script web apps and add-ons can be

deployed "anyone with the link" or shared org-wide; AppSheet apps can be shared to external users or entire domains. Uncontrolled sharing is the fastest path from a helpful internal tool to a data-leakage incident.

1. **App and script access itself (can users even build/run these).** At the platform level, the

Google Admin console controls whether Apps Script projects and AppSheet apps can access Workspace data at all, whether they must be **trusted/limited/blocked**, and — for AppSheet — whether sign-in, external sources, and external sharing are permitted. This is the outermost guardrail and the one IT owns directly.

How governance channels it

The model layers three controls so that speed and safety coexist:

• **Admin guardrails (platform).** In the Google Admin console, IT sets **app access control** for

Apps Script (trusted/limited/blocked, OAuth-scope restriction for high-risk Gmail/Drive scopes) and the **AppSheet security setting** or **AppSheet governance policies** (require sign-in, restrict external data sources, restrict external sharing, control who can mark an app as deployed, and data region). These are the non-negotiable floor.

- **Approval & review pattern (process).** A lightweight, tiered review: low-risk automations

(personal-scope, internal-only) self-serve; medium/high-risk automations (broad scopes, shared, or touching regulated data) go through a named reviewer against a checklist before they are marked deployed. AppSheet's *"only certain people can mark apps as deployed"* policy and Apps Script's deployment control make this enforceable, not just aspirational.

- **Environment & sharing standards (structure).** Named environments — **Build** (draft/dev),

Pilot (a scoped OU/group), and **Production** — with agreed sharing defaults (internal-only unless approved), owner-of-record on every artifact, and a rollback path. Reuse is explicit: shared libraries/templates for Apps Script, and a small set of blessed AppSheet templates and connected-source patterns, so builders start from a governed baseline instead of a blank canvas.

Deployment model

Like every solution in this library, deployment follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for any production change:

- **Assess** the current estate — inventory existing Apps Script projects and AppSheet apps, their

scopes, owners, and sharing (the Discovery script).

- **Configure** the admin guardrails and the review/environment standard.
- **Pilot** on a named OU/group, proving builders can still ship and that guardrails hold.
- **Validate** against conformance gates (the Validation script) before expanding.
- **Expand** to the wider organization once the pilot passes.

What this solution delivers

Seven guides (overview, requirements, implementation, testing, troubleshooting, best practices, and deliverables summary) plus five supporting PowerShell scripts — a read-only **Discovery** inventory, a **Validation** guardrail-conformance check, a **Report** on automation coverage and risk, a default-`-WhatIf` **Remediation** helper, and a **Set-ScopeTargeting** scope helper. Together they turn "we have Apps Script and AppSheet somewhere out there" into a governed, inventoried, and reviewed capability that the business can use at speed.

> ***Bottom line:*** *Business-led workflow automation that is fast *and* safe — Apps Script and AppSheet governed with guardrails, approvals, and standards — not shadow IT.*

Requirements

Before configuring Apps Script and AppSheet governance, confirm each prerequisite below. These are in addition to the shared **Global Requirements — Google Workspace & Gemini Solutions** (a provisioned tenant with a super-admin, a pilot OU/group and change control, and engaged security/data-governance stakeholders). Each item lists **Description**, **Why required**, **How to verify**, and **How to implement if missing**.

> ***Vendor boundary.*** *Google Admin console and AppSheet console paths below are authored from Google's public documentation and are current as of authoring; menu labels move over time, so **verify each path in your tenant**. Edition-gated capabilities (for example OAuth high-risk scope restriction and AppSheet governance policies) are flagged where they apply.*

1. Super-admin (or delegated) access to the Google Admin console

• **Description:** An account with the privileges to change **Security → Access and data control → API controls **(app access control) and** Apps → Google Workspace → AppSheet** settings.

• **Why required:** The platform guardrails — Apps Script app access/OAuth control and the AppSheet security setting/governance policies — are set here.

• **How to verify:** Sign in to **admin.google.com**; confirm you can open **Security → Access and data control → API controls **and** Apps → Google Workspace**.

• **How to implement if missing:** Have a super-admin grant the **Services** and **Security** admin privileges, or perform the changes with a super-admin.

2. AppSheet administration access and edition awareness

• **Description:** Access to manage AppSheet for the organization — the **AppSheet Admin Console** (now available to any admin with sufficient privileges) and, for policy-based governance, an **AppSheet Enterprise Plus** account.

• **Why required:** The single **AppSheet Core security setting** (in the Google Admin console) disables external sharing, external data sources, external automation email, external API, and webhooks for all users at once. **Granular governance policies** — *require sign-in, restrict external sources, restrict sharing to a domain, restrict who can mark an app deployed, data region* — require **Enterprise Plus**. Know which lever you have.

• **How to verify:** In the Google Admin console open **Apps → Google Workspace → AppSheet**; open the **AppSheet Admin Console**; check the AppSheet license type on your org.

• **How to implement if missing:** For all-org lockdown, the Core security setting is sufficient. For per-condition governance policies, obtain Enterprise Plus for the builders/teams in scope.

3. A data-governance baseline (data classification + connected-source inventory)

• **Description:** A basic data-classification scheme (what is public/internal/confidential/regulated) and a starting inventory of the **connected data sources** automations touch (Sheets, Drive, Cloud SQL, third-party sources).

• **Why required:** Guardrails are meaningless without knowing which automations reach sensitive data. The review model tiers on data sensitivity and OAuth-scope breadth.

• **How to verify:** Confirm a classification scheme exists and that Discovery output (see the Implementation guide) can be mapped to it.

• **How to implement if missing:** Adopt a minimal three-tier classification and run the Discovery

script to seed the connected-source/scope inventory before writing policy.

4. High-risk OAuth-scope restriction capability (edition-gated)

• **Description:** The Admin-console ability to **restrict access to high-risk OAuth scopes** (Gmail and Drive) while still allowing non-high-risk scopes — available on **Enterprise, Education Standard, or Education Plus**.

• **Why required:** This is the tightest data-access guardrail for Apps Script: it blocks apps from obtaining high-risk Gmail/Drive scopes unless the app is explicitly **trusted**.

• **How to verify:** In **Security** → **Access and data control** → **API controls** → **App access control**, select a Google service (for example Gmail) and confirm you can set restricted access to high-risk scopes.

• **How to implement if missing:** If your edition doesn't offer high-risk scope restriction, rely on app access control (trusted/limited/blocked) plus the review model to constrain scopes.

5. A named review model (roles + checklist)

• **Description:** A defined **approval/review model** — who reviews an automation before it is marked deployed, the risk tiers (self-serve vs reviewed), and the checklist they apply (scope breadth, data sensitivity, sharing, owner-of-record).

• **Why required:** Guardrails set the floor; the review model catches the medium/high-risk builds that clear the floor but still need a human decision. AppSheet's *"only certain people can mark an app as deployed"* policy and Apps Script deployment control make it enforceable.

• **How to verify:** Confirm named reviewers and a written checklist exist and are agreed with the business.

• **How to implement if missing:** Define two or three risk tiers and a one-page review checklist (see the Best Practices guide) and name at least one reviewer before the pilot.

6. A pilot OU/group and change control

• **Description:** A named **pilot organizational unit or group** to apply guardrails to first, and a **change-control** process for the org-wide settings (app access control and AppSheet security affect production access to data and apps).

• **Why required:** App access control and AppSheet security changes can block users from apps they rely on; piloting on a scoped OU proves the design before fleet-wide rollout, and change control protects production.

• **How to verify:** Confirm a pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before changing org-wide settings.

*Deployment across this solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for every production change.*

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

Implementation

This guide sets the Apps Script and AppSheet governance in the order that keeps production safe: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**. Configure the platform admin guardrails first (they are the floor), then define the review pattern and environments, then pilot on a scoped OU before expanding. Every org-wide change runs under change control with a rollback noted.

> ***Verify paths in your tenant.*** *Admin console and AppSheet console labels change over time. The paths below are current from Google's public documentation; confirm before clicking, and stage settings changes (they can take up to 24 hours to fully propagate, though usually faster).*

Phase 0 — Assess (inventory the estate)

Before changing any setting, know what exists.

1. Run `GWAUTO-Discovery.ps1` to inventory Apps Script projects and AppSheet apps — owners, OAuth scopes, connected data sources, and sharing exposure — into a baseline inventory.

1. Map each discovered automation to your data-classification scheme (Requirement 3): which touch confidential/regulated data, which are shared externally, which use broad OAuth scopes.

1. Record an **owner of record** for each artifact. Anything with no owner is the first governance backlog item.

Gate: you have a written inventory with owner, scope breadth, data sensitivity, and sharing for every discovered automation.

Phase 1 — Configure the admin guardrails (platform floor)

1a. Apps Script app access control (OAuth / data access)

Control which apps — including internal Apps Script projects — can access Workspace data.

1. In the Google Admin console, go to **Security** → **Access and data control** → **API controls**.

2. Open **App access control** (also labelled *Manage third-party app access*). Apps you have not configured as **Trusted**, **Limited**, **Specific Google data**, or **Blocked** are treated as **unconfigured**.

1. Set the **unconfigured apps** posture deliberately. Options, from most open to most closed:

• *Allow users to access any third-party apps* (default — too open for a governed estate).

• *Allow only apps that request basic Sign-in-with-Google profile info.*

• *Don't allow users to access any third-party apps until configured.*

Choose the most closed posture the business can tolerate; a common governed default is to block unconfigured apps and explicitly **trust** approved ones.

1. For **internal apps** (your own Apps Script projects), decide whether to check **Trust internal apps**. Untrusted internal apps can access only *unrestricted* services — a useful default that forces broad-scope internal automations through review before they are trusted.

1. **Restrict high-risk OAuth scopes** (edition-gated — Enterprise / Education Standard/Plus). For **Gmail** and **Drive**, set access so users can grant *non-high-risk* scopes but **high-risk scopes are restricted** unless the app is explicitly trusted. This is the tightest data-access guardrail for Apps Script.

Rollback: app access control settings are reversible — record the prior posture per service before changing so you can restore it.

1b. AppSheet governance (sharing / external data / sign-in / deploy control)

Pick the lever that matches your edition.

All editions — the Core security setting (all-or-nothing):

1. In the Google Admin console, go to **Apps** → **Google Workspace**.
2. Select **AppSheet Core license security settings**.
3. Set **Turn on AppSheet Core security**. When on, it disables, for all users: app sharing with **external** users, **external** data-source connections, automation emails to **external** recipients, **external** API integration, and **webhooks**. This is a fast, blunt floor when you don't have Enterprise Plus.

AppSheet Enterprise Plus — granular governance policies:

1. Open the **AppSheet Admin Console** (from **Apps** → **Google Workspace** → **AppSheet**, or the **Admin** → **Policies** area of the AppSheet editor).

1. Define **governance policies** as *condition + severity (Error/Warning) + stage (edit / deploy / always)*. Start with these:

• **Require sign-in** — every app must require users to sign in (condition on `[AuthRequired] = true`), severity **Error** at deploy.

• **Restrict external sharing** — apps can only be shared to your domain(s); block external sharing, severity **Error**.

• **Restrict external data sources** — constrain which connected sources are permitted.

• **Control who can deploy** — *only certain people can mark apps as deployed* (this is what makes the review model enforceable).

• **Data protection** — for example, data cannot be deleted through an app where that is a rule.

1. Apply policies at the right **level** — **organization** (highest precedence, org-wide), **team**, or **individual**. Where org and team policies overlap, AppSheet applies the **most restrictive**.

1. **Data region** (Enterprise Plus) — if you have data-residency requirements, set the AppSheet **app creation region** so data is stored/processed in the required geography.

Rollback: governance policies can be set to **Warning** (surface, don't block) during pilot, then raised to **Error** once builders have adapted — a built-in soft rollback.

1c. Apps Script sharing & deployment standard

Apps Script has no single "sharing" admin toggle equivalent to AppSheet's; sharing is governed through app access control (1a) plus a standard builders follow:

- **Web apps / add-ons** default to **execute-as the deploying user, access limited to the domain** — never "anyone, even anonymous" unless explicitly approved and reviewed.
- **Trusted deployments** (add-ons published org-wide) go through review and are added to the **trusted** app list in app access control, not left unconfigured.

Phase 2 — Define the approval/review pattern and environments

2a. Review pattern (tiered)

Tier automations by risk so speed is preserved where it's safe:

- **Tier 0 — self-serve:** personal-scope, internal-only, no confidential data → builder ships, no review, but still appears in Discovery/inventory.
- **Tier 1 — reviewed:** shared beyond the builder, or moderate scope, or internal-confidential data → one named reviewer checks the checklist before it is **marked deployed**.
- **Tier 2 — reviewed + stakeholder sign-off:** broad OAuth scopes, external sharing, or regulated data → reviewer plus data-governance/security sign-off.

The checklist (see Best Practices) covers: scope breadth, connected-source sensitivity, sharing posture, owner-of-record, and rollback. AppSheet's deploy-control policy and Apps Script deployment control make "marked deployed" a real gate for Tiers 1–2.

2b. Environments

Standardize three environments so nothing ships straight from a builder's draft to production:

- **Build** — draft/dev; personal or team scope; no production data unless masked.
- **Pilot** — the named pilot OU/group; real guardrails applied; the review model exercised.
- **Production** — org-wide (or the approved audience); guardrails at **Error**; owner-of-record and rollback recorded.

2c. Reuse baseline

Give builders a governed starting point so they don't reinvent risk:

- **Apps Script:** shared libraries and vetted script templates (auth patterns, least-scope helpers).
- **AppSheet:** a small set of blessed app templates and approved connected-source patterns.

Phase 3 — Pilot

1. Apply the admin guardrails (Phase 1) scoped to the **pilot OU/group** where possible (app access control and AppSheet policies can target OUs/teams).

1. Have pilot builders build and deploy through the review pattern and environments.

2. Run `GWAUTO-Validation.ps1` to confirm guardrail conformance, and `GWAUTO-Report.ps1` for coverage/risk in the pilot scope.

1. Keep AppSheet policies at **Warning** initially if builders need time to adapt; raise to **Error** once conformance is clean.

Gate (must pass to expand): builders can still ship; no unconfigured broad-scope app reaches production; external sharing is blocked (or explicitly approved); every deployed artifact has an owner and passed its tier's review. See the Testing guide for the exact gates.

Phase 4 — Validate and Expand

1. Confirm all **Validation** gates pass in the pilot (Testing guide).

2. Expand the guardrails from the pilot OU/group to the wider organization under change control, raising any Warning-stage AppSheet policies to **Error**.

1. Re-run **Discovery** → **Validation** → **Report** at the org scope to confirm the estate conforms and to establish the ongoing baseline.

1. Schedule a recurring Discovery + Report (for example monthly) so new automations are caught and reviewed rather than accumulating as shadow IT.

Rollback (any phase): restore the recorded prior app-access-control posture per service; set AppSheet policies back to **Warning** or off; revert the AppSheet Core security setting. Because the guardrails are configuration (not data changes), rollback is a settings restore, not a data recovery.

Testing

Testing here means **guardrail-conformance gates**: objective pass/fail checks that prove the governance is real, that builders can still ship, and that the pilot is safe to expand. Run these in the **Pilot** scope first; every gate must pass before expanding org-wide. Re-run after expansion to confirm the estate.

Use `GWAUTO-Validation.ps1` for the automated conformance checks and `GWAUTO-Report.ps1` for the coverage/risk view. Manual gates are verified in the Admin console / AppSheet Admin Console.

Gate 1 — App access control posture (Apps Script data access)

- **Check:** Unconfigured apps are set to the agreed governed posture (not the open default);

approved apps are explicitly **trusted**; internal-app trust is set deliberately.

• **Pass:** Unconfigured apps cannot obtain restricted-service access; only trusted apps can.

• **Fail:** Unconfigured apps still reach restricted services, or the posture is still the open default.

• **How:** Validation script flags the posture; confirm in **Security → Access and data control → API controls → App access control**.

Gate 2 — High-risk OAuth scope restriction (edition-gated)

• **Check:** For Gmail and Drive, high-risk scopes are **restricted** to trusted apps; non-high-risk scopes remain grantable.

• **Pass:** A new untrusted Apps Script project cannot obtain a high-risk Gmail/Drive scope.

• **Fail:** An untrusted project can still authorize a high-risk scope.

• **How:** Attempt authorization with a test project in the pilot OU; confirm it is blocked. (Skip with a documented note if your edition lacks this control.)

Gate 3 — AppSheet external sharing blocked

• **Check:** Apps cannot be shared to external users / outside approved domain(s) — via the Core security setting or an Enterprise Plus sharing policy.

• **Pass:** An attempt to share a pilot app externally is blocked (or the policy raises an Error).

• **Fail:** External sharing succeeds.

• **How:** Validation script flags AppSheet sharing config; confirm by test-sharing a pilot app.

Gate 4 — AppSheet external data sources / webhooks / external API restricted

• **Check:** External data-source connections, external automation email, external API, and webhooks are governed (Core security on, or Enterprise Plus policy).

• **Pass:** A pilot app cannot connect a disallowed external source or fire a disallowed webhook.

• **Fail:** A disallowed external connection succeeds.

• **How:** Validation script reports the AppSheet security/policy state; spot-check with a pilot app.

Gate 5 — Require sign-in

• **Check:** Deployed AppSheet apps require user sign-in (`[AuthRequired] = true`), enforced at deploy as an **Error** policy (Enterprise Plus) or per standard.

• **Pass:** An app without sign-in cannot be marked deployed.

- **Fail:** An anonymous-access app deploys.
- **How:** Attempt to deploy a no-sign-in pilot app; confirm it is blocked.

Gate 6 — Deploy control (review model is enforceable)

- **Check:** Only designated people can **mark an app as deployed** (AppSheet policy), and Apps Script Tier 1/2 deployments went through the review checklist.
- **Pass:** A non-reviewer cannot mark a pilot app deployed; reviewed Apps Script deployments have a recorded review.
- **Fail:** Any builder can self-deploy a Tier 1/2 artifact.
- **How:** Test with a non-reviewer account; check the review record for sampled deployments.

Gate 7 — Owner of record and environment discipline

- **Check:** Every deployed automation has an **owner of record**; nothing shipped from Build straight to Production without passing Pilot.
- **Pass:** Discovery/Report shows an owner for every deployed artifact; no ownerless production automation.
- **Fail:** Ownerless or un-piloted production automations exist.
- **How:** `GWAUTO-Report.ps1` ownership/coverage section.

Gate 8 — Builders can still ship (guardrails aren't a lockdown)

- **Check:** A pilot citizen developer can build and deploy a compliant Tier 0/1 automation end-to-end within the guardrails.
- **Pass:** A compliant automation ships without an exception request.
- **Fail:** Compliant work is blocked, indicating over-tight guardrails.
- **How:** Have a pilot builder complete a real, compliant workflow build.

Conformance summary

Record each gate as **Pass / Fail / N-A (edition)** with evidence (screenshot, Validation output, Report section). The pilot **expands only when Gates 1, 3–7 pass** and Gate 8 confirms builders can still ship; Gate 2 is *Pass or documented N-A*. Re-run the full set after org-wide expansion, then on the recurring Discovery cadence.

Troubleshooting

Common failure modes when standing up Apps Script and AppSheet governance, and how to resolve them. Each entry gives the **symptom**, the **likely cause**, and the **fix**. When a change involves org-wide

settings, apply the fix under change control and test on the pilot OU first.

Settings changed but nothing seems different

- **Symptom:** You changed app access control or an AppSheet setting, but behavior hasn't changed.
- **Likely cause:** Propagation delay. Admin console changes can take **up to 24 hours** to fully apply (usually faster).
- **Fix:** Wait and re-test; confirm you saved at the correct **OU/group scope** (a setting applied at a parent OU may be overridden by a child OU). Re-run `GWAUTO-Validation.ps1` after propagation.

Builders suddenly can't run apps they relied on

- **Symptom:** After tightening app access control, users are blocked from an Apps Script add-on or third-party app they need.
- **Likely cause:** The app was **unconfigured** and your unconfigured-apps posture now blocks it, or internal-app trust is off and the app needs a restricted service.
- **Fix:** Explicitly **trust** the specific, reviewed app in **Security → Access and data control → API controls → App access control** rather than loosening the whole posture. For your own reviewed Apps Script projects, add them to trusted (or check *Trust internal apps* if appropriate). This is why the design pilots on a scoped OU first.

Apps Script authorization / scope errors after restriction

- **Symptom:** A script throws an authorization error, or a user can't grant a scope it needs.
- **Likely cause: High-risk OAuth scope restriction** (Gmail/Drive) is blocking a high-risk scope for an untrusted app.
- **Fix:** Decide deliberately — either **narrow the script's scope** to a non-high-risk scope (preferred; e.g. `drive.file` instead of full Drive), or **trust** the app after review so it may request the high-risk scope. Don't disable the restriction org-wide to fix one script.

AppSheet policy isn't enforcing

- **Symptom:** You wrote an AppSheet governance policy but apps still violate it.
- **Likely cause:** (1) The policy is at **Warning** stage (surfaces, doesn't block) rather than **Error**; (2) it's applied at the wrong **level** (org vs team vs individual); or (3) you're on **AppSheet Core**, which does not support per-condition governance policies at all.
- **Fix:** Raise the policy to **Error** at the correct **stage** (edit/deploy/always) and **level** (remember org and team combine to the **most restrictive**). If you're on Core, use the **AppSheet Core security setting** (all-or-nothing) instead, and scope Enterprise Plus to the teams that need policies.

Core security setting is too blunt

- **Symptom:** Turning on the AppSheet Core security setting blocked a legitimate external data source or webhook a team needs.
- **Likely cause:** The Core setting is **all-or-nothing** — it disables external sharing, external sources, external automation email, external API, and webhooks together.
- **Fix:** If you need to permit one of these for a specific team, you need **Enterprise Plus** governance policies (granular) for that team rather than the blunt Core toggle; otherwise the capability stays off for everyone.

"Anyone can mark apps deployed" — review model not enforced

- **Symptom:** Builders self-deploy Tier 1/2 automations without review.
- **Likely cause:** The **deploy-control** policy (*only certain people can mark apps as deployed*) isn't configured, or you're on Core (no policy support).
- **Fix:** On Enterprise Plus, add the deploy-control policy and name the reviewers. For Apps Script, enforce review through the deployment/trust step and the review checklist before an add-on is trusted org-wide.

Discovery/Validation returns nothing or errors

- **Symptom:** `GWAUTO-Discovery.ps1` or `GWAUTO-Validation.ps1` returns empty or errors on auth.
- **Likely cause:** Missing admin privileges/scopes for the tooling, wrong target OU/group, or the Workspace API/AppSheet Admin data not accessible to the running account.
- **Fix:** Run as an account with the required admin privileges; confirm the target scope with `GWAUTO-Set-ScopeTargeting.ps1 -WhatIf`; verify API access. See each script's `.DESCRIPTION` header for required scopes/tools.

Data-residency requirement not met

- **Symptom:** AppSheet data is stored/processed outside the required region.
- **Likely cause:** The AppSheet **app creation region** wasn't set (Enterprise Plus feature).
- **Fix:** Set the required **data region** in AppSheet (Enterprise Plus). Note existing apps may need to be recreated in-region — confirm current behavior in AppSheet's data-residency docs for your tenant.

Remediation would change more than expected

- **Symptom:** `GWAUTO-Remediation.ps1 -WhatIf` previews changes you didn't intend.

• **Likely cause:** Broad targeting, or flagged items that need a human decision (external sharing, broad scopes) rather than an automated fix.

• **Fix:** Narrow the scope with `GWAUTO-Set-ScopeTargeting.ps1`, review the `-WhatIf` output line by line, and only run without `-WhatIf` for the specific, reviewed items. The default `-WhatIf` exists precisely so nothing changes until you've confirmed the plan.

If a fix requires an org-wide setting change, stage it under change control and validate on the pilot OU before expanding — the same discipline the Implementation guide uses.

Best Practices — Safe Citizen Development

These practices keep Apps Script and AppSheet fast for the business while holding the enterprise standard. They are written for both audiences: the **citizen developer** building a workflow and the **IT/architecture** owner setting the guardrails. The governing idea: *make the safe path the easy path.*

Least scope, always

• Request the **narrowest OAuth scope** that does the job. Prefer per-file/`drive.file` over full Drive; prefer a specific Gmail scope over full mailbox access. Broad scopes are the single biggest Apps Script risk and the thing high-risk-scope restriction exists to stop.

• Treat any broad-scope request as an automatic **review trigger**, not a default.

• In AppSheet, connect only the **data the app needs** — narrow the source, don't wire the whole spreadsheet or database when a view will do.

Internal-only by default; external sharing is a decision

• Default Apps Script web apps/add-ons to **execute-as the deploying user, access limited to the domain** — never "anyone, even anonymous" without explicit approval.

• Keep AppSheet sharing **inside the approved domain(s)**; external sharing is an exception that goes through review, not a builder's unilateral choice. The Core security setting or an Enterprise Plus policy should enforce this so it isn't left to memory.

Require sign-in and know the user

• Every deployed AppSheet app should **require sign-in** (`[AuthRequired] = true`) so access is attributable and revocable. Anonymous apps have no audit trail.

Owner of record on everything

• No automation ships without a **named owner** responsible for it — the person who fixes it, answers for its data access, and retires it when it's obsolete. Ownerless automation is future shadow IT.

- When an owner leaves, reassignment is part of offboarding.

Build → Pilot → Production, never straight to prod

- Develop in **Build**, prove in **Pilot** (the scoped OU/group with real guardrails), then promote to **Production**. Don't test on live confidential data; mask or use sample data in Build.
- Record a **rollback** for every production automation — how to disable/revert it if it misbehaves.

Reuse the governed baseline

- Start from **blessed templates and shared libraries** (Apps Script auth/least-scope helpers, approved AppSheet templates and connected-source patterns) instead of a blank canvas. Reuse is faster *and* safer because the baseline already meets the standard.

Tier the review, don't gate everything

- Keep **Tier 0 self-serve** (personal, internal, non-confidential) genuinely frictionless so builders don't route around governance. Reserve review for **Tier 1** (shared/moderate) and **Tier 2** (broad scope / external / regulated). Over-gating breeds shadow IT; under-gating leaks data — tiering targets the risk.

A one-page review checklist

For any Tier 1/2 automation, the reviewer confirms:

1. **Scope** — least-privilege OAuth scopes / narrowest connected source?
2. **Data** — what classification does it touch; is that approved?
3. **Sharing** — internal-only, or approved external with a reason?
4. **Sign-in** — required (AppSheet)?
5. **Owner** — named owner of record?
6. **Rollback** — documented way to disable/revert?

If all six are clean, it may be **marked deployed** (deploy-control policy enforces this in AppSheet).

Keep the inventory alive

- Run **Discovery + Report on a recurring cadence** (for example monthly) so new automations surface and get an owner and a review tier — governance is continuous, not a one-time project.
- Use the **Report** risk view to prioritize: broad-scope + external + confidential automations get attention first.

Change under control; propagation takes time

- Org-wide app access control and AppSheet security changes affect real users' access — make them

under **change control**, on a **pilot OU first**, and remember settings can take up to 24 hours to fully propagate. Use AppSheet **Warning** stage before **Error** to give builders a grace period.

Deliverables Summary

This solution delivers everything needed to establish and operate work-transformation governance for Google Apps Script and AppSheet — the written guidance and the supporting scripts — so a competent admin/architect can run it self-guided and pilot-first.

Documentation

Guide	What it gives you
Solution Overview	The automation governance model for Apps Script + AppSheet: where the risk is (data access/OAuth scopes, sharing, app access) and how admin guardrails, an approval/review pattern, and environment/sharing standards channel it.
Requirements	Prerequisites with Description / Why / Verify / Implement — admin access, AppSheet edition awareness (Core vs Enterprise Plus), data-governance baseline, high-risk OAuth-scope capability, a named review model, and a pilot OU/change control.
Implementation	Step-by-step: configure Apps Script app access/OAuth control and AppSheet security/governance policies and data region; define the tiered approval/review pattern and the Build/Pilot/Production environments and reuse baseline; pilot; validate; expand — with rollback.
Testing	Eight guardrail-conformance gates (app access posture, high-risk scopes, external sharing, external sources/webhooks, require sign-in, deploy control, owner/environment discipline, builders-can-still-ship) with explicit pass/fail and the expand gate.
Troubleshooting	Common failure modes and fixes — broken app access, over-tight guardrails blocking builders, scope/authorization errors, AppSheet policy vs Core-setting confusion, propagation delay, edition gating.
Best Practices	Safe citizen development — least scope, internal-only by default, require sign-in, owner of record, Build→Pilot→Prod, reuse the governed baseline, tiered review, and a one-page review checklist.

Scripts (5)

All scripts carry a standard header (Synopsis / Purpose / How to run / Expected results / Notes). The three inventory/validation/report scripts are **read-only** where they read live systems; the two change-capable scripts default to `-WhatIf` (preview only) and make no change until explicitly run without it.

Script	Role	Change-safe
GWAUTO-Discovery.ps1	Inventory Apps Script projects and AppSheet apps — owners, OAuth scopes, connected sources, sharing exposure.	Read-only
GWAUTO-Validation.ps1	Check guardrail conformance against the Testing gates (app access posture, sharing, sources, sign-in, deploy control).	Read-only
GWAUTO-Report.ps1	Automation coverage and risk report — scope breadth, external sharing, data sensitivity, ownership gaps.	Read-only
GWAUTO-Remediation.ps1	Bring flagged items toward conformance (owner assignment, sharing/scope fixes). Defaults to `-WhatIf`.	`-WhatIf` default
GWAUTO-Set-ScopeTargeting.ps1	Set which OUs/groups/teams the guardrails and review model target. Defaults to `-WhatIf`.	`-WhatIf` default

Outcome

Run end-to-end, the deliverables convert an ungoverned Apps Script/AppSheet estate into an inventoried, guardrailed, reviewed capability: builders ship fast within a governed baseline; IT holds the standard; and the business gets workflow automation that is fast **and** safe — not shadow IT.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

GWAUTO-Discovery.ps1

Purpose: Reads an export of your Apps Script projects and AppSheet apps and emits a governance inventory: owner of record, OAuth scopes, connected data sources, sharing exposure, and the assigned risk tier. This is the Assess-phase baseline for the whole solution. Read-only — it reads an export you supply; it changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWAUTO-Discovery.ps1 -InventoryCsv C:\Temp\automation-inventory.csv Build the export from the Google Admin console (App access control / OAuth app inventory) and the AppSheet Admin Console (apps, owners, sharing), or from the Workspace Admin SDK / OAuth token reports for your-domain. Read-only Workspace admin visibility is enough to produce it. CSV columns: Platform,ArtifactId,Name,Owner,Scopes,DataSources,Sharing,Environment,Tier (Platform = AppsScript or AppSheet; Scopes/DataSources semicolon-separated; Sharing = Internal, Domain, External, or Anyone.) EXPECTED RESULTS A per-artifact table plus counts by platform, sharing exposure, and risk tier. Artifacts with no owner, broad scopes, or external/anyone sharing are flagged. No system is changed. Pass = the inventory reconciles with your Admin/AppSheet consoles and every artifact has an owner. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\GWAUTO-Discovery.ps1 -InventoryCsv C:\Temp\automation-inventory.csv Build the export from the Google Admin console (App access control / OAuth app inventory) and the AppSheet Admin Console (apps, owners, sharing), or from the Workspace Admin SDK / OAuth token reports for your-domain. Read-only Workspace admin visibility is enough to produce it. CSV columns: Platform,ArtifactId,Name,Owner,Scopes,DataSources,Sharing,Environment,Tier (Platform = AppsScript or AppSheet; Scopes/DataSources semicolon-separated; Sharing = Internal, Domain, External, or Anyone.) EXPECTED RESULTS A per-artifact table plus counts by platform, sharing exposure, and risk tier. Artifacts with no owner, broad scopes, or external/anyone sharing are flagged. No system is changed. Pass = the inventory reconciles with your Admin/AppSheet consoles and every artifact has an owner. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [string]$CsvPath)

GWAUTO-Remediation.ps1

Purpose: Turns the risk report (from GWAUTO-Report.ps1) or a findings CSV into tracked remediation actions: for each non-conformant automation — no owner, broad OAuth scope, external sharing, or missing sign-in — it drafts an action item (owner, due date, required fix) and, where you keep a governance tracker as a Microsoft List, creates the item via Microsoft Graph. Defaults to -WhatIf so nothing is created until you review the plan and re-run with -Confirm. It records the governance actions to take; it does not itself change an Apps Script/AppSheet setting. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\GWAUTO-Remediation.ps1 -FindingsCsv C:\Temp\automation-risk.csv Create the action items in a governance Microsoft List (after review): powershell -ExecutionPolicy Bypass -File .\GWAUTO-Remediation.ps1 -FindingsCsv C:\Temp\automation-risk.csv -SiteId your-site-id -ListId your-actions-list-id -Confirm Graph mode requires the Microsoft Graph PowerShell SDK and the Sites.ReadWrite.All scope. FindingsCsv columns: Platform,Name,Owner,Sharing,RiskScore,RiskBand (extra columns ignored). EXPECTED RESULTS In

-WhatIf: the remediation actions that WOULD be opened, with owner, fix, and due date — no change. With -Confirm: a CSV action plan is written, and (in Graph mode) one list item is created per finding. Pass = every non-conformant automation has an owned, dated action. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$FindingsCsv, [string]$SiteId, [string]$ListId, [int]$DueInDays = 10, [string]$OutDir = '.')

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\GWAUTO-Remediation.ps1 -FindingsCsv C:\Temp\automation-risk.csv Create the action items in a governance Microsoft List (after review): powershell -ExecutionPolicy Bypass -File .\GWAUTO-Remediation.ps1 -FindingsCsv C:\Temp\automation-risk.csv -SiteId your-site-id -ListId your-actions-list-id -Confirm Graph mode requires the Microsoft Graph PowerShell SDK and the Sites.ReadWrite.All scope. FindingsCsv columns: Platform,Name,Owner,Sharing,RiskScore,RiskBand (extra columns ignored). EXPECTED RESULTS In -WhatIf: the remediation actions that WOULD be opened, with owner, fix, and due date — no change. With -Confirm: a CSV action plan is written, and (in Graph mode) one list item is created per finding. Pass = every non-conformant automation has an owned, dated action. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$FindingsCsv, [string]$SiteId, [string]$ListId, [int]$DueInDays = 10, [string]$OutDir = '.')

GWAUTO-Report.ps1

Purpose: Turns the automation inventory (from GWAUTO-Discovery.ps1) into a coverage-and-risk report: how many automations are governed vs ungoverned, a per-artifact risk score (broad scope + external sharing + confidential data + no owner), and the top items to remediate first. Read-only — it reads the inventory export you supply and changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWAUTO-Report.ps1 -InventoryCsv C:\Temp\automation-inventory.csv -CsvPath C:\Temp\automation-risk.csv InventoryCsv columns: Platform,ArtifactId,Name,Owner,Scopes,DataSources,Sharing,Environment,Tier Optionally add a Sensitivity column (Public/Internal/Confidential/Regulated) to sharpen scoring; absent, sensitivity contributes 0. Read-only Workspace/AppSheet admin visibility produces it. EXPECTED RESULTS Coverage counts (governed vs ungoverned, by environment and tier), a ranked risk table, and the highest-risk artifacts to address first. Optional CSV export of the scored inventory. No system is changed. Pass = coverage and risk are understood and the top risks have named owners/actions. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\GWAUTO-Report.ps1 -InventoryCsv C:\Temp\automation-inventory.csv -CsvPath C:\Temp\automation-risk.csv InventoryCsv columns: Platform,ArtifactId,Name,Owner,Scopes,DataSources,Sharing,Environment,Tier Optionally add a Sensitivity column (Public/Internal/Confidential/Regulated) to sharpen scoring; absent, sensitivity contributes 0. Read-only Workspace/AppSheet admin visibility produces it. EXPECTED RESULTS Coverage counts (governed vs ungoverned, by environment and tier), a ranked risk table, and the highest-risk artifacts to address first. Optional CSV export of the scored inventory. No system is changed. Pass = coverage and risk are understood and the top risks have named owners/actions. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [string]$CsvPath)

GWAUTO-Set-ScopeTargeting.ps1

Purpose: Creates (idempotently) the security groups that scope who the Apps Script/AppSheet guardrails and review model apply to: 'Automation - Pilot Builders - Scope' (the pilot OU/group the guardrails are applied to first) and 'Automation - Reviewers - Deploy Approvers' (the named people allowed to mark apps deployed / approve Tier 1-2 builds). Optionally seeds each group from a CSV of user principal names. Defaults to -WhatIf so nothing is created until you review. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\GWAUTO-Set-ScopeTargeting.ps1 Create groups and seed rosters (after review): powershell -ExecutionPolicy Bypass -File .\GWAUTO-Set-ScopeTargeting.ps1 -PilotBuildersCsv C:\Temp\pilot.csv -ReviewersCsv C:\Temp\reviewers.csv -Confirm Each CSV needs a UserPrincipalName column. Requires the Microsoft Graph PowerShell SDK and the Group.ReadWrite.All and User.Read.All scopes. Use these groups to target app access control and AppSheet governance-policy scope in the Google Admin/AppSheet console for your-domain. EXPECTED RESULTS In -WhatIf: the groups and seed actions that WOULD run — no change. With -Confirm: both groups exist (created if missing, reused if present) and are seeded from any CSV supplied; group object IDs are printed. Pass = both scope groups exist with intended rosters. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([string]$PilotBuildersCsv, [string]$ReviewersCsv)

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\GWAUTO-Set-ScopeTargeting.ps1 Create groups and seed rosters (after review): powershell -ExecutionPolicy Bypass -File .\GWAUTO-Set-ScopeTargeting.ps1 -PilotBuildersCsv C:\Temp\pilot.csv -ReviewersCsv C:\Temp\reviewers.csv -Confirm Each CSV needs a UserPrincipalName column. Requires the Microsoft Graph PowerShell SDK and the Group.ReadWrite.All and User.Read.All scopes. Use these groups to target app access control and AppSheet governance-policy scope in the Google Admin/AppSheet console for your-domain. EXPECTED RESULTS In -WhatIf: the groups and seed actions that WOULD run — no change. With -Confirm: both groups exist (created if missing, reused if present) and are seeded from any CSV supplied; group object IDs are printed. Pass = both scope groups exist with intended rosters. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([string]$PilotBuildersCsv, [string]$ReviewersCsv)

GWAUTO-Validation.ps1

Purpose: Evaluates the automation inventory (from GWAUTO-Discovery.ps1) and your recorded admin-guardrail posture against the Testing-guide conformance gates: app access posture, external sharing, external data sources, require sign-in, deploy control, and owner/environment discipline. Emits per-gate Pass/Fail with the offending artifacts. Read-only — it evaluates data you supply and changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWAUTO-Validation.ps1 -InventoryCsv C:\Temp\automation-inventory.csv -PostureJson C:\Temp\guardrail-posture.json InventoryCsv columns: Platform,ArtifactId,Name,Owner,Scopes,DataSources,Sharing,Environment,Tier PostureJson keys (record your Admin/AppSheet console settings for your-domain), e.g.: { "UnconfiguredApps":"Blocked", "HighRiskScopeRestriction":true, "AppSheetExternalSharing":"Blocked", "AppSheetExternalSources":"Blocked", "RequireSignIn":true, "DeployControl":true } Read-only Workspace/AppSheet admin visibility is enough to populate both inputs. EXPECTED RESULTS A per-gate result (Pass / Fail / N-A) with the artifacts that fail each gate. The pilot expands only when the required gates pass. No system is changed. Exit code 0 = all required gates pass; 3 = one or more required gates failed. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #>

[CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [Parameter(Mandatory)][string]$PostureJson)

How to run: powershell -ExecutionPolicy Bypass -File .\GWAUTO-Validation.ps1 -InventoryCsv C:\Temp\automation-inventory.csv -PostureJson C:\Temp\guardrail-posture.json InventoryCsv columns: Platform,ArtifactId,Name,Owner,Scopes,DataSources,Sharing,Environment,Tier PostureJson keys (record your Admin/AppSheet console settings for your-domain), e.g.: { "UnconfiguredApps":"Blocked", "HighRiskScopeRestriction":true, "AppSheetExternalSharing":"Blocked", "AppSheetExternalSources":"Blocked", "RequireSignIn":true, "DeployControl":true } Read-only Workspace/AppSheet admin visibility is enough to populate both inputs. EXPECTED RESULTS A per-gate result (Pass / Fail / N-A) with the artifacts that fail each gate. The pilot expands only when the required gates pass. No system is changed. Exit code 0 = all required gates pass; 3 = one or more required gates failed. .NOTES White-label — the repeatable way. Read-only where it reads live systems. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$InventoryCsv, [Parameter(Mandatory)][string]$PostureJson)